

Product Design

Team no. 35

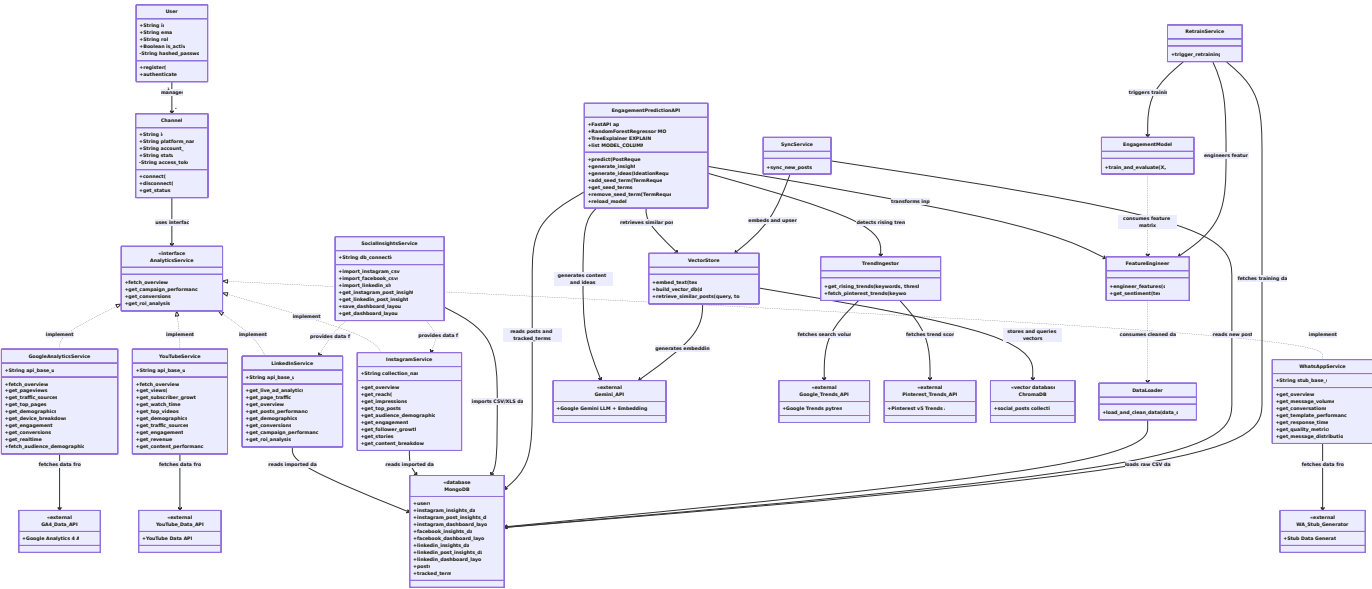
Devshree Vyas, Kaushik Yadala, Kushagra Agrawal, Sian Shinjo, Srinivas Kolla

This is a "living document", meaning its content and format will grow with the implementation of the project. It captures key project concepts and documents important design decisions.

Design Model

The Club Artizen Social Media Analytics Dashboard follows a modular, service-oriented architecture divided into four tiers: a Next.js frontend, a FastAPI backend, a MongoDB database, and a dedicated ML microservice for engagement prediction, trend analysis, and AI-powered content generation. The backend isolates platform-specific logic through dedicated Route/Service pairs per social channel. A manual data-import pipeline supports CSV/XLS ingestion for Instagram, Facebook, and LinkedIn. Dashboard layouts are user-specific and persisted per channel. The ML service operates as an independent FastAPI application, providing a Random Forest engagement predictor with SHAP explainability, a RAG pipeline (ChromaDB + Gemini) for trend-aware strategic insights, and an AI content ideation engine grounded in historical top-performing posts.

Class Diagram



Class Descriptions

Class / Module	Description
----------------	-------------

User	Class state: Maintains user profile state including <code>id</code> , <code>email</code> , <code>role</code> (Admin / Co-Founder / Marketing Team), <code>is_active</code> boolean, and <code>hashed_password</code> for security. Uses Pydantic models (<code>UserBase</code> , <code>UserCreate</code> , <code>UserDB</code> , <code>UserResponse</code>) and a custom <code>PyObjectId</code> adapter for MongoDB ObjectId serialization. Class behavior: The <code>auth_routes</code> module provides <code>signup()</code> for onboarding new users, <code>login()</code> to authenticate and generate JWT tokens, and <code>get_current_user()</code> / <code>get_current_admin_user()</code> dependency-injected guards for secure endpoint access.
Channel	Class state: Stores tracking configuration per integrated platform (LinkedIn, Instagram, Facebook, WhatsApp, Google Analytics, YouTube). Keeps <code>platform_name</code> , <code>account_id</code> , <code>account_name</code> , <code>status</code> (Connected/Disconnected), and encrypted <code>access_token</code> / <code>refresh_token</code> fields. Class behavior: Provides <code>connect()</code> for OAuth/token exchange, <code>disconnect()</code> for teardown, and <code>get_status()</code> to ping connection health.
AnalyticsService	Class state: Acts as a base interface. Minimal internal state, maintains shared behavior signatures. Class behavior: Defines common analytics operations: <code>fetch_overview()</code> , <code>get_campaign_performance()</code> , <code>get_conversions()</code> , and <code>get_roi_analysis()</code> . All platform-specific services implement this interface.
GoogleAnalyticsService	Class state: Holds the GA4 property ID and service-account credentials path. Connects to the GA4 Data API (external). Class behavior: Inherits <code>AnalyticsService</code> methods and implements platform-specific endpoints: <code>get_pageviews()</code> , <code>get_traffic_sources()</code> , <code>get_top_pages()</code> , <code>get_device_breakdown()</code> , <code>get_engagement()</code> , <code>get_realtime()</code> , and <code>fetch_audience_demographics()</code> .
YouTubeService	Class state: Holds YouTube Data API key and channel ID. Connects to the YouTube Data API v3 (external). Class behavior: Inherits <code>AnalyticsService</code> methods and implements <code>get_views()</code> , <code>get_subscriber_growth()</code> , <code>get_watch_time()</code> , <code>get_top_videos()</code> , <code>get_demographics()</code> , <code>get_traffic_sources()</code> , <code>get_engagement()</code> , <code>get_revenue()</code> , and <code>get_content_performance()</code> .

LinkedInService	Class state: Holds LinkedIn API credentials (client ID/secret, org URN, access token, ad-account URN, API version). Reads imported data from MongoDB. Class behavior: Inherits AnalyticsService methods and implements <code>get_overview()</code>, <code>get_posts_performance()</code>, <code>get_demographics()</code>, <code>get_page_traffic()</code>, <code>get_live_ad_analytics()</code>, and <code>get_roi_analysis()</code>. Includes specialized conversion / campaign-performance models.
InstagramService	Class state: Operates primarily through saved MongoDB documents imported via CSV. Reads from MongoDB. Class behavior: Inherits AnalyticsService methods and provides <code>get_overview()</code>, <code>get_reach()</code>, <code>get_impressions()</code>, <code>get_top_posts()</code>, <code>get_audience_demographics()</code>, <code>get_engagement()</code>, <code>get_follower_growth()</code>, <code>get_stories()</code>, and <code>get_content_breakdown()</code>.
WhatsAppService	Class state: Configuration-driven; connects to a Stub Data Generator for WhatsApp Business metrics. Class behavior: Inherits AnalyticsService methods and implements <code>get_overview()</code>, <code>get_message_volume()</code>, <code>get_conversations()</code>, <code>get_template_performance()</code>, <code>get_response_time()</code>, <code>get_quality_metrics()</code>, and <code>get_message_distribution()</code>.
SocialInsightsService	Class state: Manages MongoDB collections for manual platform-data imports (Instagram CSV, Facebook CSV, LinkedIn XLS) and per-channel dashboard layouts. Class behavior: Exposes routes under <code>/manual/*</code> for bulk CSV/ZIP import, XLS import for LinkedIn, per-post insights retrieval, channel-level aggregated metrics, and dashboard layout CRUD for all three platforms. Provides data for InstagramService and LinkedInService.
Widget	Class state: Stores per-user dashboard widget instances including <code>widget_type</code>, <code>configuration</code> (arbitrary JSON payload), and <code>dashboard_position</code> (grid coordinates). Class behavior: Supports CRUD for adding, removing, and rearranging widgets on dashboards; enables per-channel dashboard layout persistence for Instagram, Facebook, and LinkedIn.

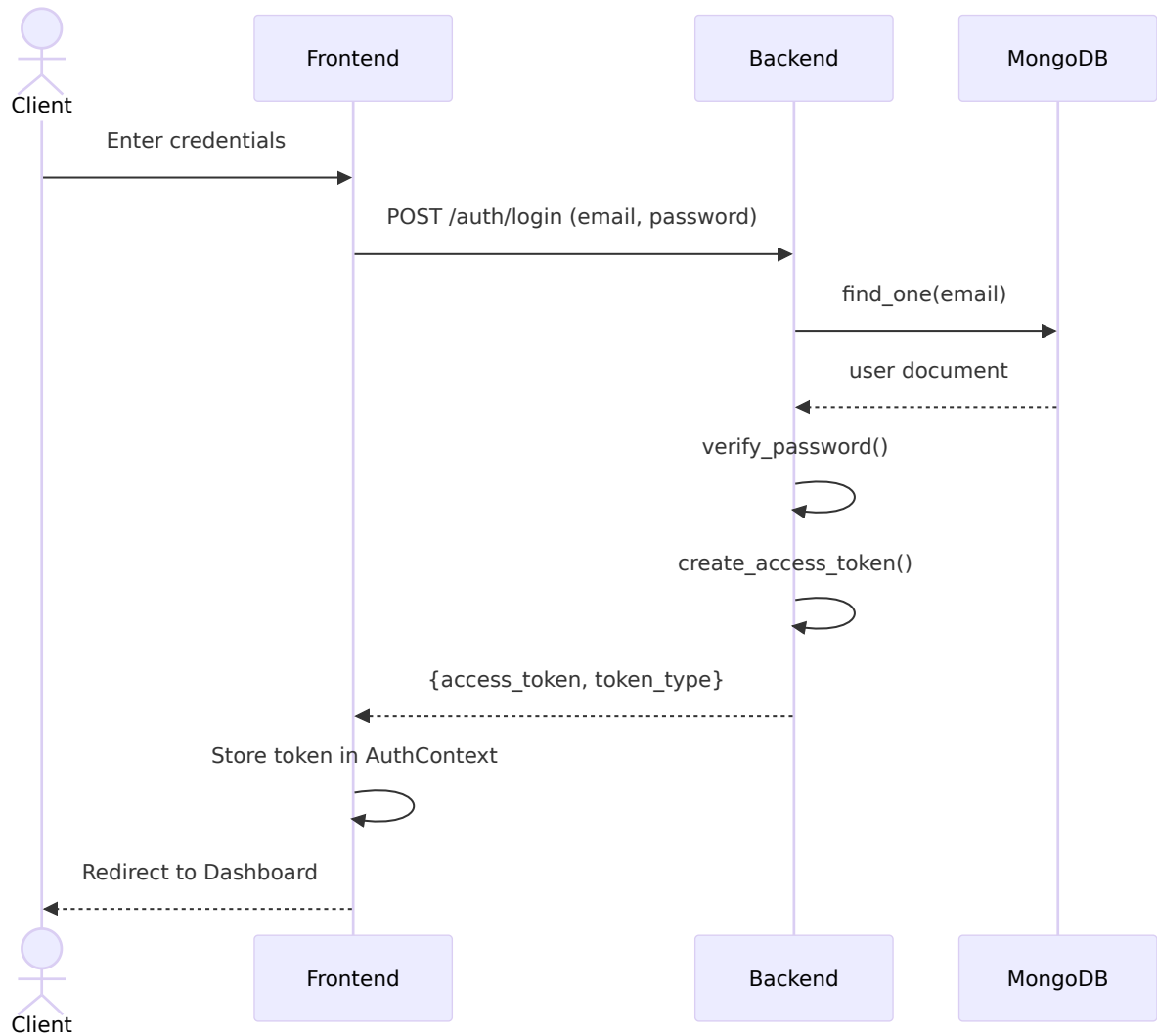
Settings (Config)	Class state: Central configuration singleton loaded from <code>.env</code> via <code>pydantic-settings</code>. Holds credentials and URIs for GA4, YouTube, LinkedIn, MongoDB, JWT secrets, and CORS origins. Class behavior: Exposes computed properties such as <code>ga_credentials_available</code>, <code>yt_credentials_available</code>, <code>linkedin_credentials_available</code>, <code>linkedin_live_api_ready</code>, and <code>cors_origins_list</code> to gate feature availability at runtime.
Database	Class state: Holds a singleton reference to the <code>AsyncIOMotorClient</code> connection. Class behavior: Provides <code>get_db()</code> as a FastAPI dependency for injecting the database handle into route handlers. Lifecycle is managed via the FastAPI <code>lifespan</code> context manager.
Security	Class state: Maintains a <code>passlib CryptContext</code> configured for bcrypt hashing. Class behavior: Exposes <code>verify_password()</code>, <code>get_password_hash()</code>, and <code>create_access_token()</code> (JWT creation with configurable expiry).
EngagementPredictionAPI	Class state: Holds references to the loaded <code>RandomForestRegressor</code> model, its <code>TreeExplainer</code> (SHAP), and the ordered <code>MODEL_COLUMNS</code> list — all loaded into RAM once at startup via a FastAPI <code>lifespan</code> context manager. Connects to MongoDB for post data and tracked seed terms, and to the Gemini API client for generative content. Class behavior: Exposes <code>/predict</code> (engagement prediction with SHAP explanations), <code>/generate_insight</code> (RAG-powered trend analysis combining Google Trends, Pinterest, ChromaDB vector search, and Gemini LLM), <code>/generate_ideas</code> (content ideation grounded in historical top performers), CRUD endpoints for <code>/terms/*</code> (seed term management), and <code>/admin/reload_model</code> (hot-reloading model artifacts without restarting the server). Performs cold-start detection on boot: if model artifacts are missing, triggers an automated retraining pipeline before proceeding.
DataLoader	Class state: Stateless utility module; resolves CSV file paths relative to <code>data/raw/posts</code>. Class behavior: Uses Polars lazy execution (<code>scan_csv</code>) to load all CSVs from the raw data directory, filters out rows with null or zero <code>Reach</code>, calculates <code>Like_Rate</code> (Likes / Reach) as the target variable, selects only the columns needed for downstream feature engineering, and materializes the result via <code>.collect()</code>.

FeatureEngineer	Class state: Stateless transformation module. Class behavior: Accepts a cleaned Polars DataFrame and produces a numerical feature matrix <code>X</code> and target series <code>y</code>. Engineers text-based features (<code>hashtag_count</code> , <code>word_count</code> , <code>sentiment</code> via <code>TextBlob</code>), temporal features (cyclical sine/cosine encoding of <code>hour</code> and <code>weekday</code> to capture time-of-day and day-of-week periodicity), and performs one-hot encoding of <code>Post type</code>. Supports both training mode (returns <code>y</code>) and inference mode (returns <code>y=None</code> when <code>Like_Rate</code> is absent).
EngagementModel	Class state: Stateless training module; writes model artifacts to <code>models/</code> directory. Class behavior: Trains a constrained <code>RandomForestRegressor</code> (100 trees, <code>max_depth=3</code> , <code>min_samples_leaf=5</code>) on the feature matrix. Evaluates using MAE and R^2 on a 20% held-out test set. Extracts SHAP-based global feature importance rankings. Serializes the trained model and its column schema to <code>.joblib</code> files for later hot-loading by the API.
VectorStore	Class state: Manages a persistent ChromaDB collection (<code>social_posts</code>) stored on disk under <code>chroma_db/</code>. Connects to the Gemini Embedding API (<code>gemini-embedding-001</code>) for text vectorization. Class behavior: Provides <code>embed_text()</code> to generate dense embeddings, <code>build_vector_db()</code> to batch-embed and upsert a historical DataFrame into ChromaDB, and <code>retrieve_similar_posts()</code> to perform semantic nearest-neighbor search (cosine similarity) and return matching post captions with metadata (post type, like rate).
TrendIngestor	Class state: Stateless utility module; connects to Google Trends (via <code>pytrends</code>) and the Pinterest v5 Trends API. Class behavior: <code>get_rising_trends()</code> queries Google Trends for a list of seed keywords, computes 7-day momentum as the percentage change between recent and historical search volume averages, and returns keywords exceeding a configurable threshold. <code>fetch_pinterest_trends()</code> queries the Pinterest API for keyword trend scores and returns high-momentum results. Both functions include rate-limit-aware delays and error handling.

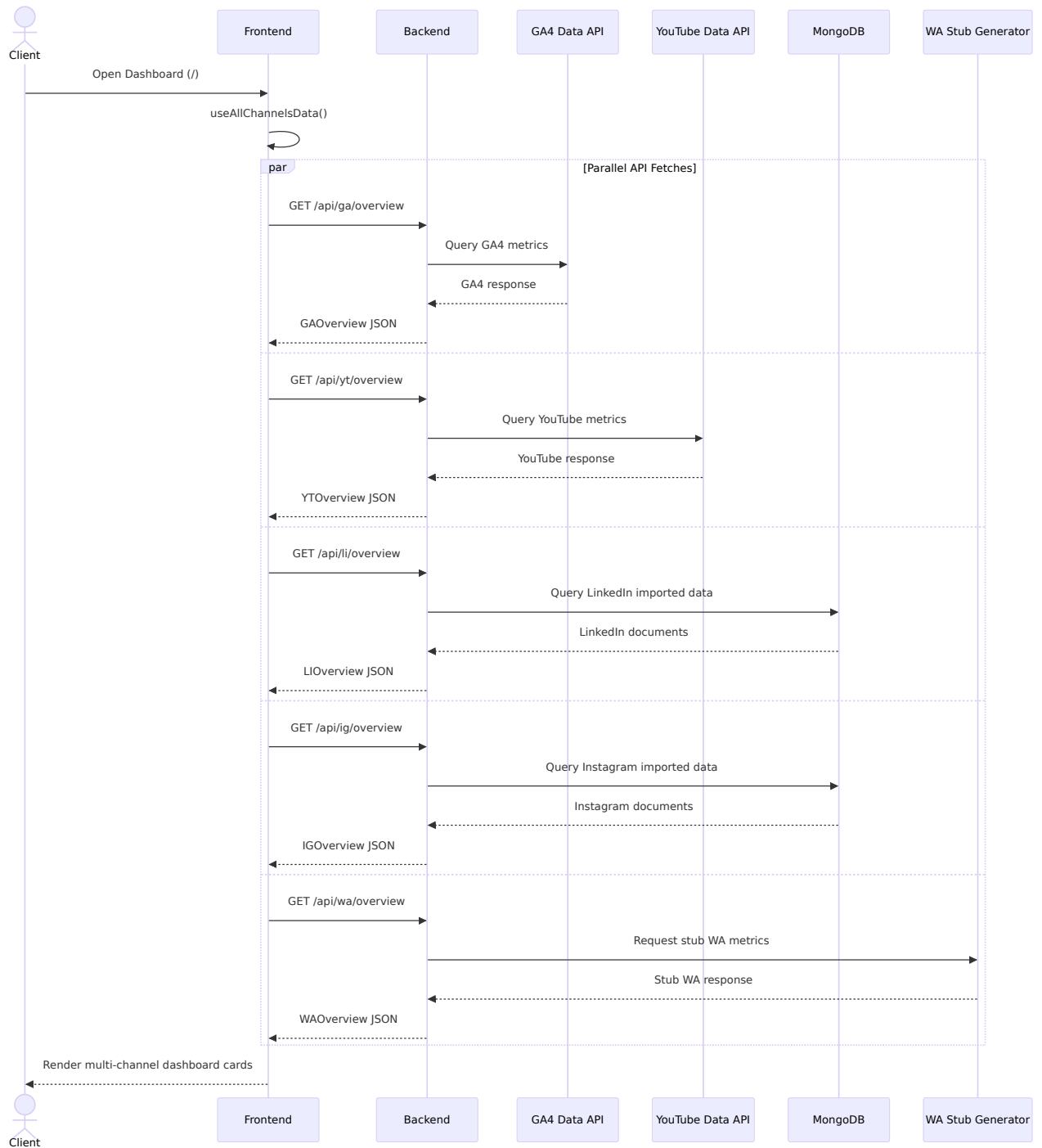
SyncService	Class state: Connects to both MongoDB (source of truth for posts) and ChromaDB (vector store). Class behavior: <code>sync_new_posts()</code> performs an incremental sync — it queries ChromaDB for existing post IDs, fetches only new posts from MongoDB that are not yet embedded, generates embeddings via <code>VectorStore.embed_text()</code>, and upserts them into ChromaDB with metadata. Supports a configurable <code>MAX_SYNC_EMBEDS</code> limit to prevent rate-limiting during large syncs. Returns the pre-sync ChromaDB count to enable cold-start detection by the API.
RetrainService	Class state: Connects to MongoDB to fetch the latest post data. Class behavior: <code>trigger_retraining()</code> pulls all posts from MongoDB, converts them from snake_case Mongo field names to the title-case schema expected by <code>FeatureEngineer</code>, runs feature engineering, trains a new model via <code>EngagementModel.train_and_evaluate()</code>, and writes updated <code>.joblib</code> artifacts to disk. Requires a minimum of 50 posts to proceed (guards against training on insufficient data).

Sequence Diagrams

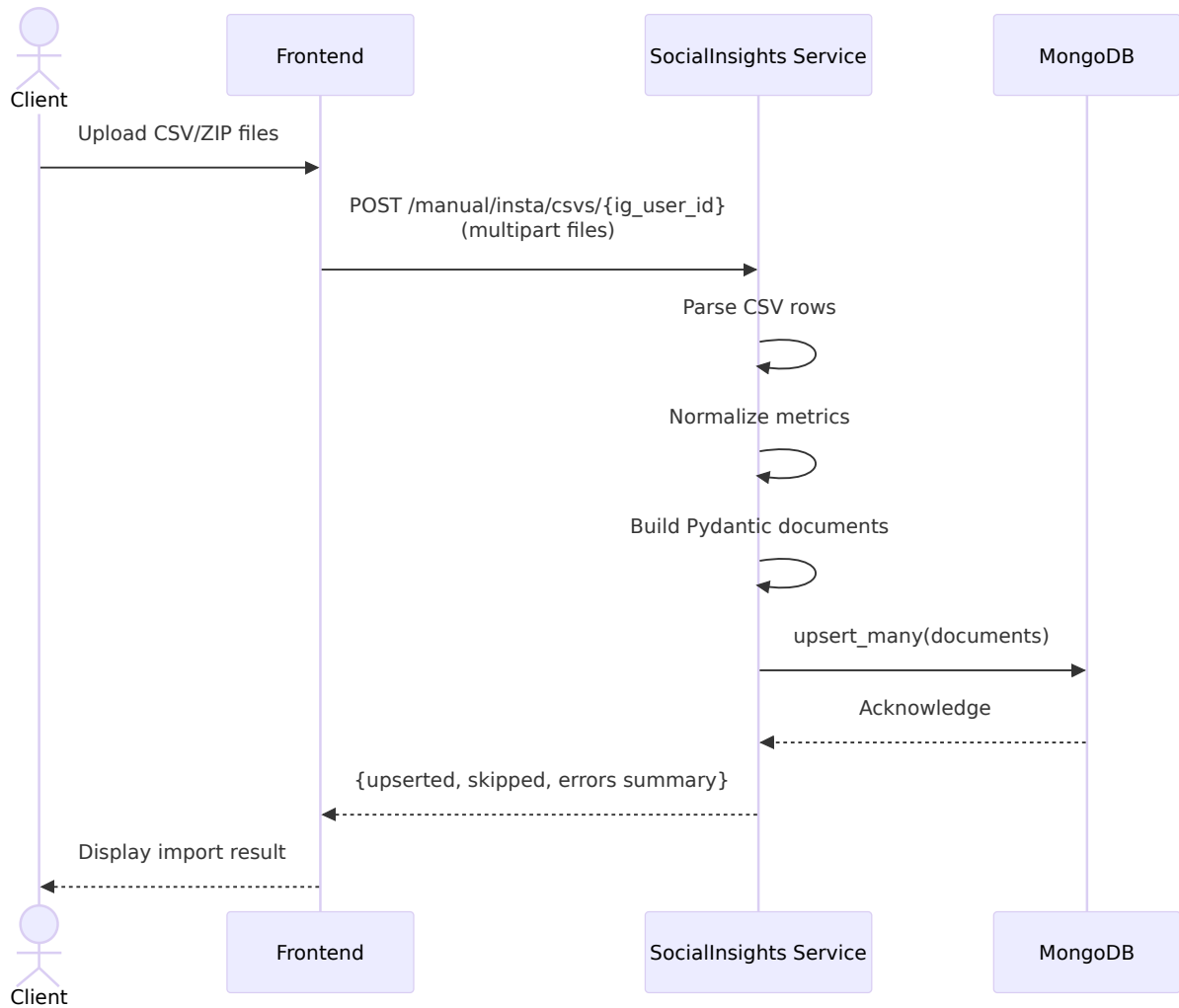
Use Case 1: User Login & Authentication Flow



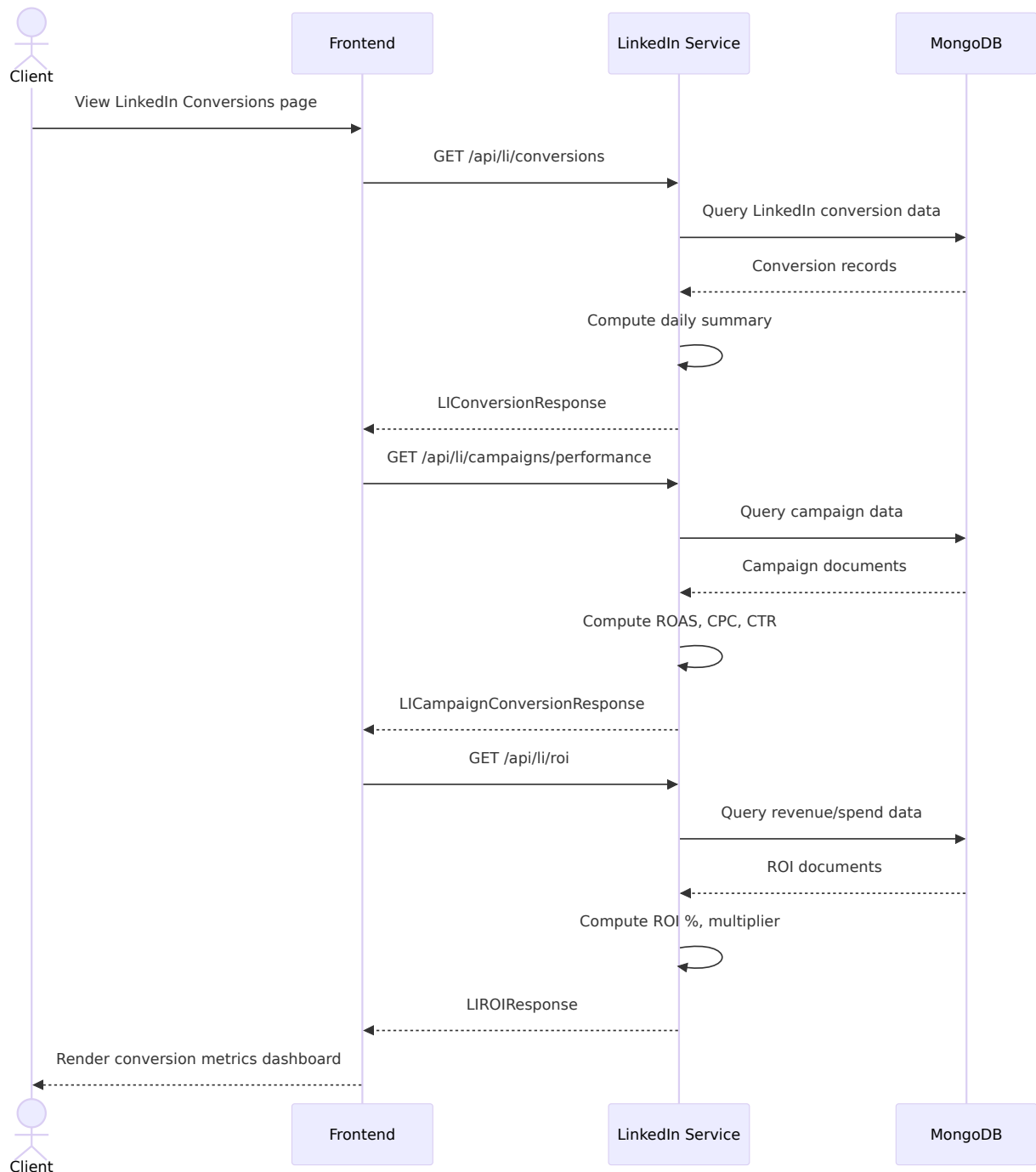
Use Case 2: Main Dashboard Multi-Channel Overview



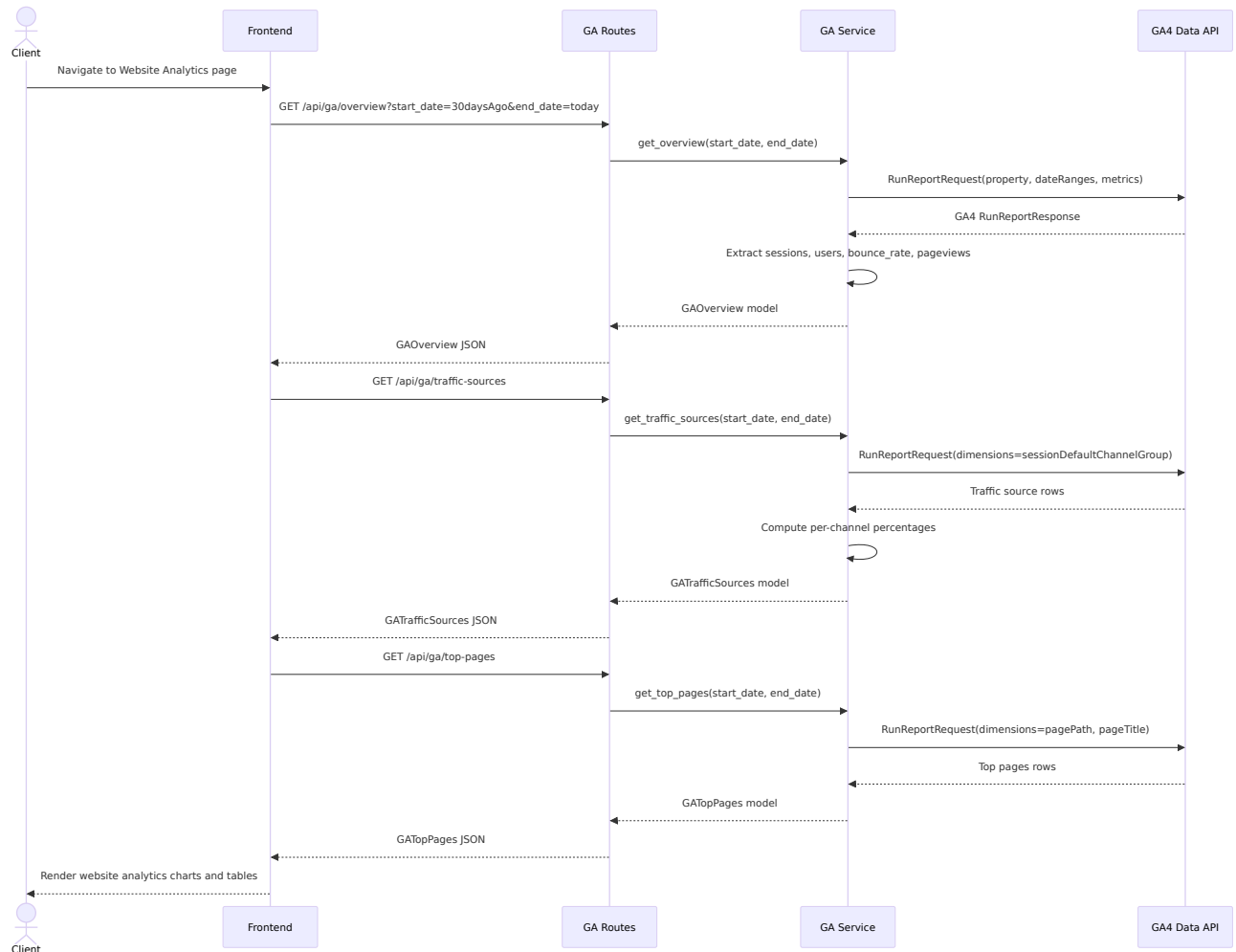
Use Case 3: Manual CSV/XLS Import for Social Insights



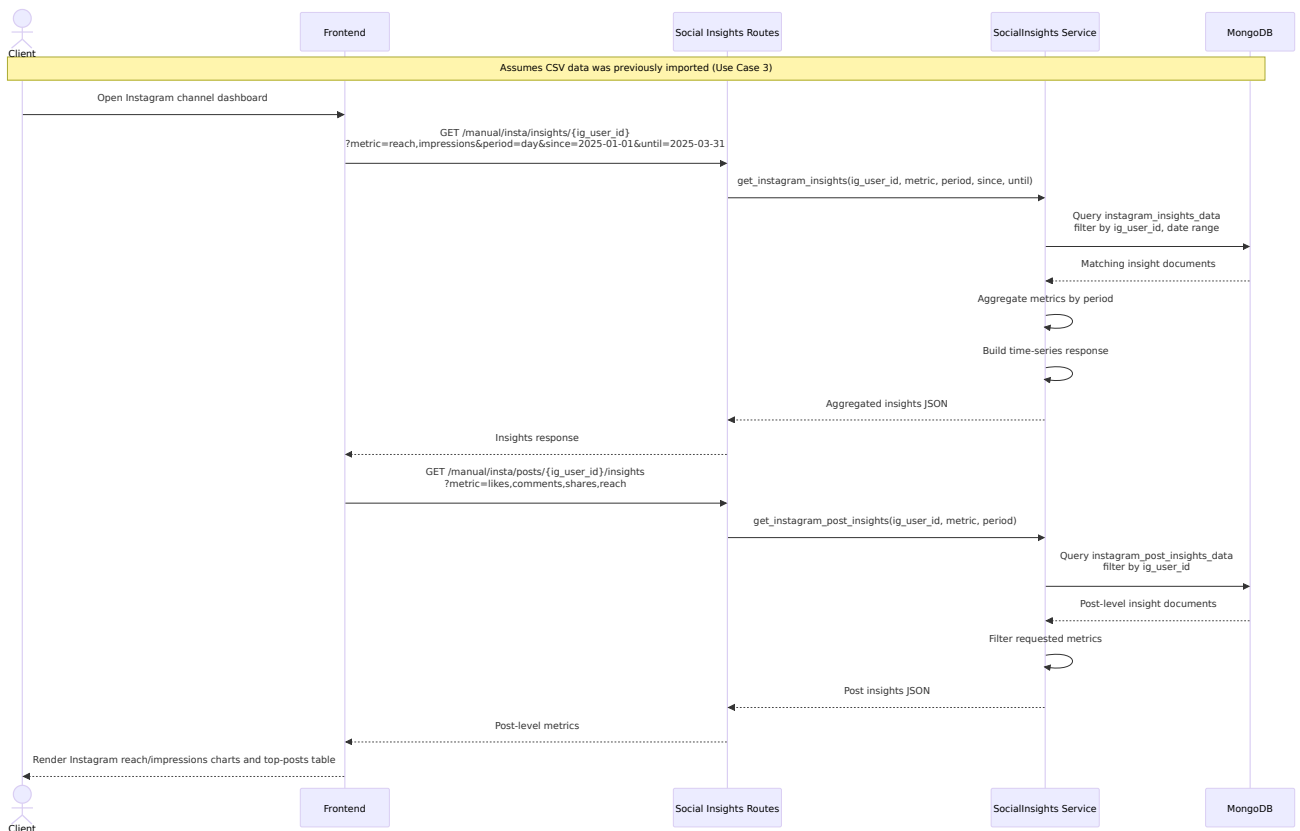
Use Case 4: LinkedIn Conversion & ROI Analysis



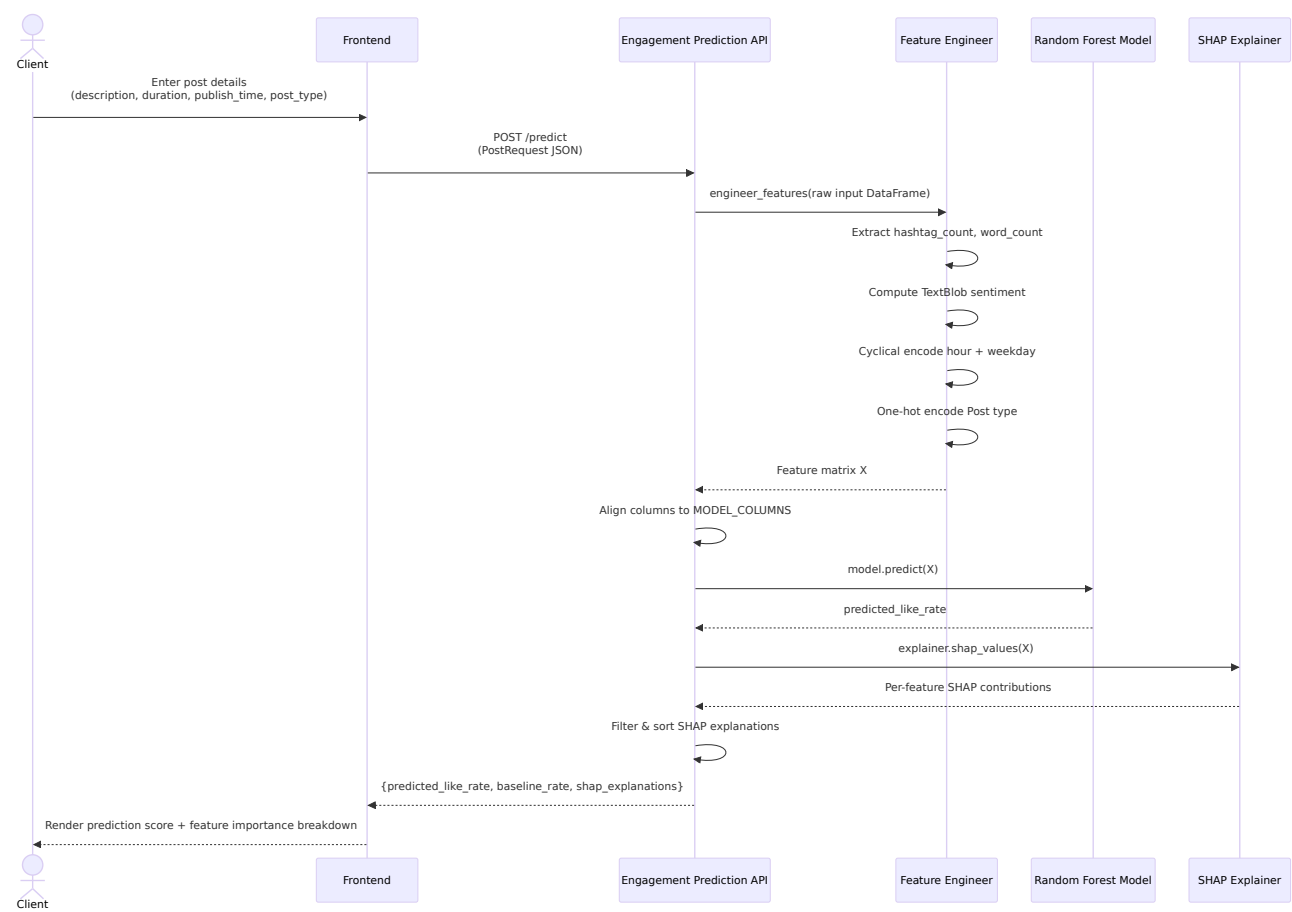
Use Case 5: Fetching Google Analytics Website Metrics



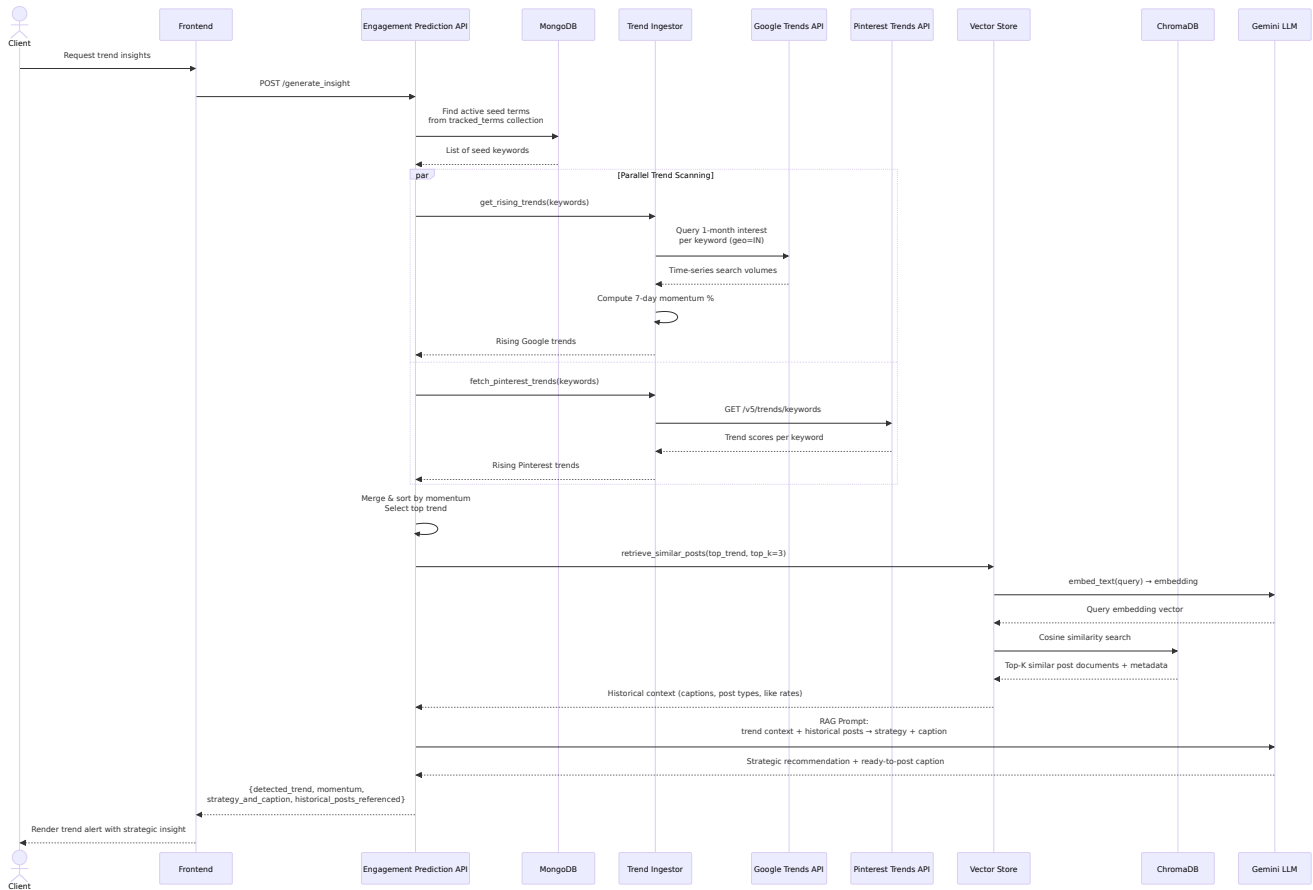
Use Case 6: Querying CSV-Imported Instagram Metrics



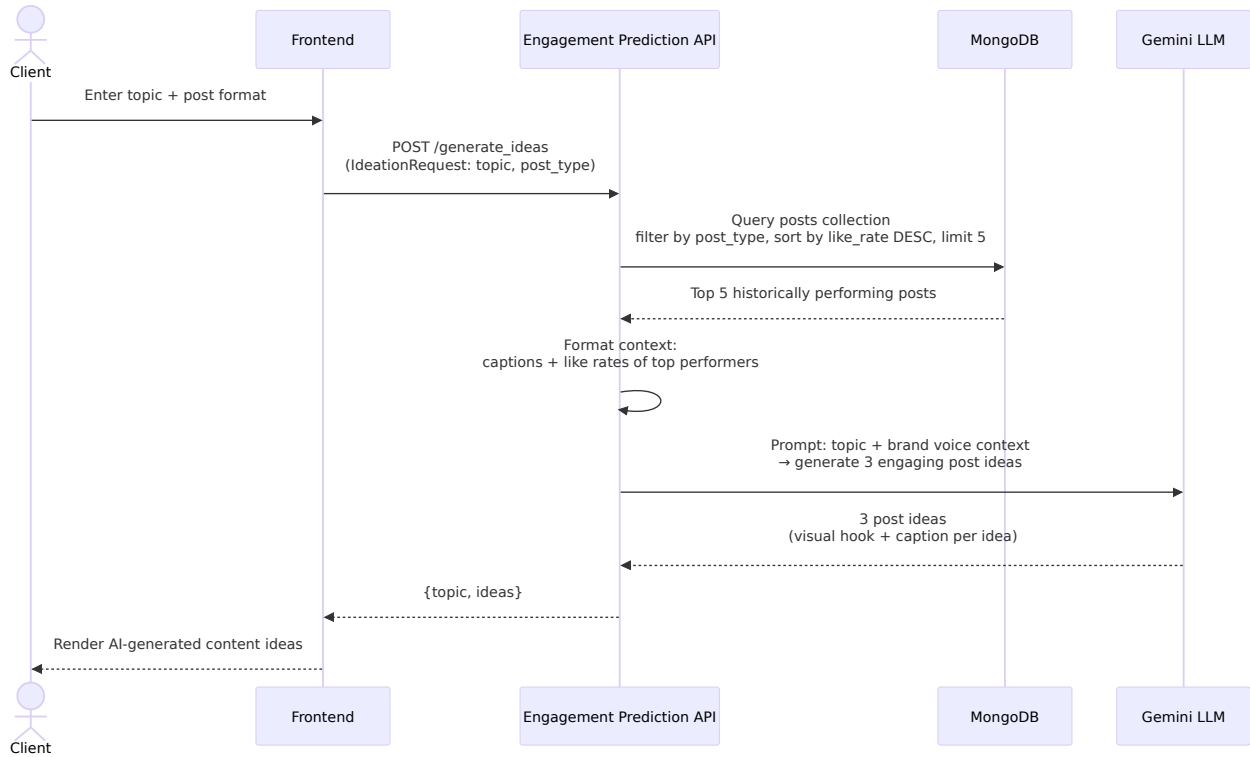
Use Case 7: Predicting Engagement for a New Post (ML Service)



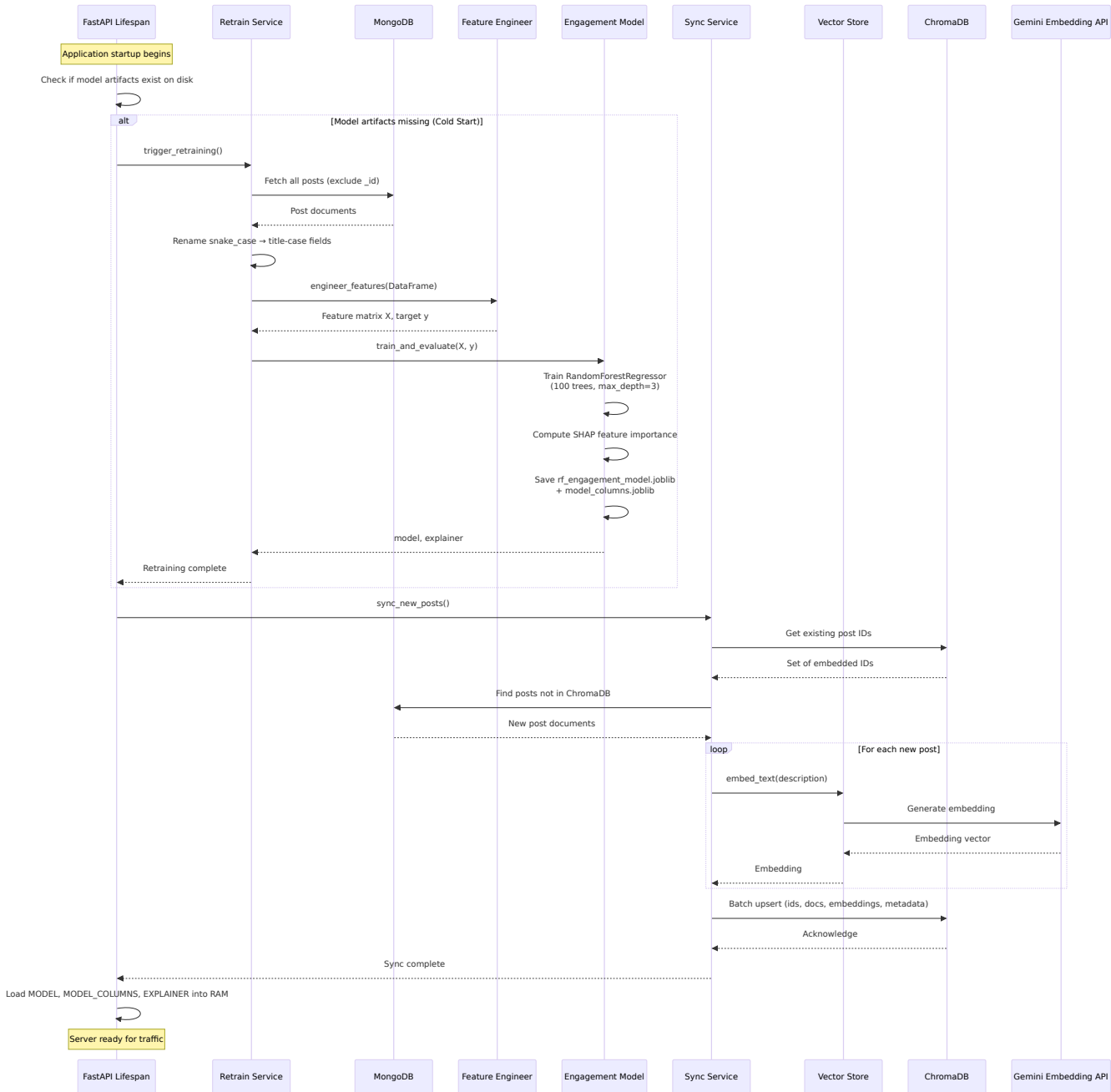
Use Case 8: RAG-Powered Trend Insight Generation (ML Service)



Use Case 9: AI-Powered Content Ideation (ML Service)



Use Case 10: ML Service Cold Start & Model Retraining



Design Rationale

As development proceeded, our team investigated options across database technologies, application frameworks, and isolation boundaries. The final design decisions (and some rejected alternatives) are outlined below.

1. Using a Modular Route/Service Architecture

- **Alternative Considered:** A monolithic backend where all platform logic, data parsing, and API orchestration live in a single module.
- **Rationale for Selection:** We adopted a strict Route → Service split per platform channel (e.g., `ga_routes.py` → `ga_service.py`, `li_routes.py` → `li_service.py`, etc.). Each route module

is responsible only for HTTP concerns (request validation, response serialization), while the paired service module encapsulates all business logic, external API calls, and data transformation. This yields:

- **Testability:** Services can be unit-tested by mocking external API clients or the database, independent of HTTP constructs.
- **Reusability:** If a new dashboard widget requires the same data, we call the service method directly without refactoring the router.
- **Independent scaling:** Service modules with heavy computation (e.g., `social_insights_service.py` at ~90 KB) can be profiled and optimized in isolation.

2. Choosing FastAPI / MongoDB Stack

- **Alternative Considered:** Relational databases (PostgreSQL) paired with a traditional MVC framework (Django/Flask).
- **Rationale for Selection:** Social media channel metrics present highly nested, irregular, and platform-specific data shapes (e.g., standard Google Analytics metadata differs massively from metadata received by Instagram and LinkedIn APIs). MongoDB affords us a flexible NoSQL schema using Pydantic configurations to serialize/deserialize data safely. FastAPI supports asynchronous behavior out of the box, which is critically important given that the majority of our platform aggregates data from high-latency external APIs. Async capabilities (via `motor` async MongoDB driver) prevent thread blocking when fetching data across multiple remote platforms simultaneously.

3. Manual CSV/XLS Import Pipeline vs. Live API Only

- **Alternative Considered:** Building a standalone stub server that returns randomly generated data for Instagram, Facebook, and LinkedIn, serving as a proof of concept to demonstrate the dashboard's capabilities without real platform data.
- **Rationale for Selection:** Our client was unable to acquire the necessary API keys and OAuth credentials for Instagram, Facebook, and LinkedIn due to platform approval processes and access restrictions. Rather than blocking development or relying on purely synthetic random data (which would not reflect realistic metric distributions or temporal patterns), we introduced a manual-import pipeline (`SocialInsightsService`) that accepts bulk CSV uploads (Instagram, Facebook) and XLS workbooks (LinkedIn) — formats that the client can readily export from each platform's native analytics dashboard. The service normalizes the heterogeneous column schemas into a unified document model and upserts records into MongoDB keyed by `post_id + ig_user_id / li_org_id`. This approach gives the client full benefit of the dashboard's visualization, aggregation, and comparison features using their own real data, without requiring live API credentials. A stub server was rejected because random data, while useful for integration testing, would not allow the client to draw actionable insights from their actual social media performance. The pipeline also supports ZIP-archive uploads for convenience when importing large export folders.

4. Per-Channel Dashboard Layout Persistence

- **Alternative Considered:** A single, global dashboard layout shared across all users and channels.
- **Rationale for Selection:** Different team members (Admin, Co-Founder, Marketing Team) care about different KPIs. We introduced per-channel layout collections (`instagram_dashboard_layouts`, `facebook_dashboard_layouts`, `linkedin_dashboard_layouts`) keyed by (`channel_user_id`, `dashboard_user_id`). The frontend uses `react-grid-layout` for drag-and-drop widget rearrangement, and the layout state is persisted via PUT/GET endpoints. This enables each user to maintain their own customized view per channel without affecting other users.

5. Next.js 16 + React 19 Frontend with Shadcn/UI

- **Alternative Considered:** A plain React SPA with Create React App, or a Vue.js frontend.
- **Rationale for Selection:** Next.js provides server-side rendering capabilities, file-system based routing, and built-in API route support. React 19's concurrent features improve user experience during parallel data fetches. We chose `shadcn/ui` (built on Radix UI) for accessible, unstyled component primitives paired with Tailwind CSS v4, delivering a polished, modern dashboard aesthetic with minimal custom CSS. `Recharts` provides declarative chart components that integrate naturally with React's component model.

6. Unified SocialInsightsService vs. Per-Platform Import Services

- **Alternative Considered:** Creating separate, independent import services for each platform (e.g., `InstagramImportService`, `FacebookImportService`, `LinkedInImportService`), each with its own CSV/XLS parsing logic, MongoDB collection management, and dashboard-layout persistence.
- **Rationale for Selection:** We chose to consolidate all manual-import and dashboard-layout logic into a single `SocialInsightsService` module. Although this results in a larger service file (~90 KB), it provides significant advantages:
 - **Shared parsing infrastructure:** CSV parsing, ZIP extraction, date normalization, and metric validation logic is written once and reused across Instagram, Facebook, and LinkedIn imports, reducing code duplication.
 - **Consistent dashboard-layout API:** The layout persistence endpoints (`get_*_dashboard_layout`, `save_*_dashboard_layout`) share the same `InstagramDashboardWidgetInstance` model across all three platforms, ensuring uniform widget configuration and serialization.
 - **Single entry point for testing:** All import edge cases (malformed CSV, missing columns, duplicate `post_id` handling) are tested through one service module, simplifying the test suite (`test_social_insights_routes.py`, `test_social_insights_posts_service.py`).
 - **Easier onboarding of new platforms:** Adding a new CSV/XLS-backed channel requires only extending the existing service with a new set of handlers, rather than scaffolding an entirely

new service, route, and test module.

7. JWT-Based Authentication with Role-Based Access Control

- **Alternative Considered:** Session-based authentication with cookies, or delegating to a third-party auth provider (Auth0, Firebase Auth).
- **Rationale for Selection:** JWT tokens are stateless and pair naturally with a decoupled frontend/backend architecture. We implemented `python-jose` for token encoding/decoding and `passlib[bcrypt]` for password hashing. The `get_current_user()` dependency provides a clean guard pattern for protected endpoints, while `get_current_admin_user()` adds role-based access control. The frontend `AuthContext` manages token lifecycle (storage, refresh, logout).

8. Pydantic Models for API Contract Enforcement

- **Alternative Considered:** Returning raw dictionaries from service methods without schema enforcement.
- **Rationale for Selection:** We define explicit Pydantic response models for every endpoint (e.g., `GAOverview`, `YTTTopVideos`, `LIConversionResponse`, `IGTopPosts`, `WATemplatePerformance`). This approach:
 - Provides automatic request/response validation and documentation via FastAPI's OpenAPI spec.
 - Catches schema drift between the backend and frontend early.
 - Enables the frontend TypeScript type definitions to mirror the backend models, creating a strongly-typed contract across the full stack.
 - Uses `ConfigDict(extra="forbid")` on import document models to reject unexpected fields, enforcing data integrity on CSV/XLS ingestion.

9. Separate ML Microservice Architecture

- **Alternative Considered:** Embedding the ML prediction, trend analysis, and content generation logic directly within the main FastAPI backend alongside the analytics route/service modules.
- **Rationale for Selection:** The ML workload has fundamentally different resource characteristics from the analytics API: it loads a ~100 KB Random Forest model and a SHAP `TreeExplainer` into RAM at startup, performs CPU-intensive inference and SHAP value computation per request, and maintains a persistent ChromaDB vector database on disk. Isolating these into a dedicated `ml-service` microservice provides:
 - **Independent scaling:** The ML service can be deployed on a compute-optimized instance (or GPU node if upgraded to deep learning models) without affecting the I/O-bound analytics API.
 - **Decoupled deployment:** Model retraining, vector database rebuilding, and dependency updates (scikit-learn, SHAP, ChromaDB) happen in isolation — a broken model retrain does not take down the dashboard.

- Cleaner dependency graph: The ML service depends on `scikit-learn`, `shap`, `chromadb`, `google-genai`, `pytrends`, `textblob`, and `polars` — none of which are needed by the main backend. Keeping them separate avoids dependency conflicts and reduces cold-start size.

10. Random Forest with SHAP Explainability for Engagement Prediction

- **Alternative Considered:** Using a deep learning model (LSTM or Transformer-based) for engagement prediction, or a simpler linear regression baseline.
- **Rationale for Selection:** With a relatively small dataset (~hundreds of Instagram posts), a constrained Random Forest (`max_depth=3`, `min_samples_leaf=5`, 100 estimators) provides the best bias-variance trade-off. Deep learning models require orders of magnitude more data to generalize; on our dataset they would severely overfit. Linear regression, while interpretable, cannot capture non-linear interactions between features (e.g., the combined effect of posting time and content sentiment). The Random Forest:
 - Handles mixed feature types (numerical, one-hot encoded) natively.
 - Provides inherent regularization through depth and leaf-size constraints.
 - Pairs naturally with SHAP's `TreeExplainer`, which computes exact Shapley values in polynomial time for tree ensembles — enabling per-prediction explanations (e.g., "posting at 6 PM contributed +0.02 to your predicted like rate") that build user trust and actionability.

11. RAG Pipeline with ChromaDB + Gemini for Trend Insights

- **Alternative Considered:** (a) Directly prompting the LLM without any retrieval context. (b) Using a full-text keyword search (e.g., MongoDB text index) instead of semantic vector search.
- **Rationale for Selection:** A pure LLM prompt (option a) hallucinates brand-specific recommendations because it has no knowledge of the client's historical post performance. Full-text search (option b) misses semantically related content (e.g., a search for "sustainable fashion" would not match a caption about "eco-friendly khadi collection" which is thematically identical). Our Retrieval-Augmented Generation pipeline combines:
 - Gemini Embedding API (`gemini-embedding-001`) to convert post captions into dense 768-dimensional vectors.
 - ChromaDB (persistent, on-disk) as a lightweight vector store requiring no external infrastructure — ideal for a project that doesn't need the operational complexity of Pinecone or Weaviate.
 - Semantic retrieval via cosine similarity to surface the most relevant historical posts for any trending topic.
 - Grounded generation by injecting retrieved context (captions, post types, like rates) into the Gemini LLM prompt, producing strategic recommendations anchored in the client's actual performance data rather than generic social media advice.

12. Polars-First Data Pipeline with Pandas Interop

- **Alternative Considered:** Using Pandas throughout the entire ML pipeline (data loading, feature engineering, model training).
- **Rationale for Selection:** Polars offers significant advantages for the data loading and feature engineering stages: lazy execution (`scan_csv`) enables query optimization before data touches RAM, columnar operations are implemented in Rust for better performance, and the API enforces explicit type handling that catches data quality issues early. However, scikit-learn and SHAP require Pandas/NumPy arrays. Our pipeline uses Polars for all upstream processing (`DataLoader` , `FeatureEngineer`) and converts to Pandas only at the model boundary (`X_polars.to_pandas()` in `model.py` and `api.py`). This hybrid approach yields:
 - Faster data loading and transformation (critical during model retraining on growing datasets).
 - A clean separation between data-engineering code (Polars) and ML-library interop code (Pandas).
 - Future-proofing for when scikit-learn adds native Polars support.